

How Many Lines to Paint the City: Exact Edge-Cover in Temporal Graphs

Advanced Artificial Intelligence - A.L.MA. ¹

Andreas Panayi

Department of Informatics & Telecommunications - NKUA

December 2025



¹andreaspanayi8@gmail.com

1. INTRODUCTION

Problem

Informally

In many real-world systems, connections or opportunities (trips, tasks, or contacts) are only available at certain times. The goal is to plan and allocate limited resources (vehicles, crews, robots, agents) so every time-stamped connection is executed.

Formally

Given a dynamic network in which connections are available only at specific times, determine the minimum number of resources required to realise all required connections.

What this models:

- Scheduling vehicles, crews, or agents in transportation and logistics systems.
- Coordinating time-sensitive tasks in multi-agent and robotic systems.
- Planning maintenance, data-collection passes, or service calls that must happen at fixed times.
- Exact-use constraints (each time-stamped task must be performed by exactly one resource).

Practical benefits:

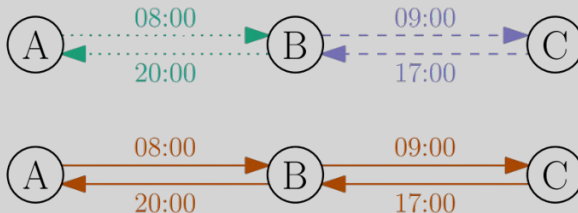
- Reduces operational cost by minimising fleet and crew sizes.
- Improves resource utilisation and decreases idle time.
- Enables robust contingency planning under temporal constraints

Relation to AI and where it appears

- AI methods (search, combinatorial optimization, scheduling) can compute or approximate minimal-resource allocations for time-dependent tasks.
- *Planning & multi-agent systems*: coordinating many agents under temporal constraints.
- *Predictive + planning pipeline*: data-driven demand prediction (e.g., forecasting future trips or requests) can feed a deterministic scheduler that solves an instance of our problem.
- *Operations research & optimization engines*: specialized solvers or heuristics used in AI/OR for vehicle routing, crew scheduling, dispatching can be applied directly.

Intuition & Example

- The figure shows scheduled trips (edges) with their timestamps for two operators. It illustrates how, when timings align, a single vehicle can serve several trips in sequence.
- Takeaway: exploiting temporal ordering can substantially reduce the number of required resources.



2. MATHEMATICAL MODELING

Definition (Temporal Graph)

A *temporal graph* $G := \langle G, \mathcal{E} \rangle$ is defined by:

- an underlying static graph (directed or undirected) $G = (V, E)$, and
- a sequence of edge-sets $\mathcal{E} = (E_1, E_2, \dots, E_{t_{\max}})$ where each $E_t \subseteq E$ represents the edges available at time t ,
- with $E = E_1 \cup E_2 \cup \dots \cup E_{t_{\max}}$.

The temporal graph can be equivalently seen as a sequence of static graphs $(G_1, G_2, \dots, G_{t_{\max}})$ with $G_t = (V, E_t)$.

Temporal Graph Example

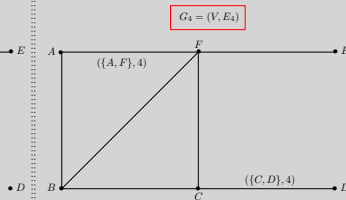
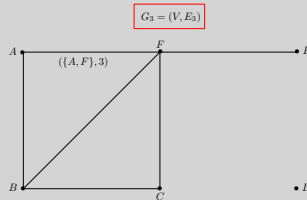
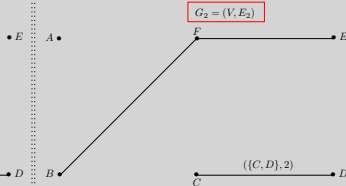
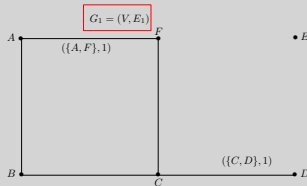
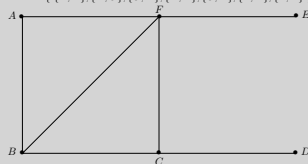
$$G_{temp} = (G, \mathcal{E})$$

$$\mathcal{E} = (E_1, E_2, E_3, E_4)$$

$$G = (V, E)$$

$$V = \{A, B, C, D, E, F\}$$

$$E = \{ \{A, B\}, \{B, C\}, \{C, D\}, \{B, F\}, \{C, F\}, \{F, E\}, \{F, A\} \}$$



Temporal journeys

- A *temporal journey* is a way to travel through the graph over time, using only edges that are present when we traverse them.
- Consecutive edges must respect time:
 - non-strict*: $t_{i+1} \geq t_i$,
 - strict*: $t_{i+1} > t_i$.
- Journeys model feasible movements of a single resource (vehicle, agent) through the dynamic network.

Temporal walks, trails, and paths

Consider a sequence

$$v_0 \xrightarrow[(t_1)]{e_1} v_1 \xrightarrow[(t_2)]{e_2} \cdots \xrightarrow[(t_\ell)]{e_\ell} v_\ell$$

where each $e_i = \{v_{i-1}, v_i\}$ (or (v_{i-1}, v_i)) and $t_i \in \lambda(e_i)$ with $t_1 \leq t_2 \leq \cdots \leq t_\ell$ (or strictly increasing in the strict case).

- *Temporal walk*: vertices and edges may repeat.
- *Temporal trail*: edges are all distinct; vertices may repeat.
- *Temporal path*: all vertices (hence edges) are distinct.

(Non-)Strict Temporal Φ Exact Edge-Cover

Definition: Temporal exact edge-cover

Let $G = \langle (V, E), \mathcal{E} \rangle$ be a temporal graph (lifetime $t_{\max}$) and let $\Phi \in \{\text{path}, \text{trail}, \text{walk}\}$.

A collection $\mathcal{J} = \{J_1, \dots, J_k\}$ of temporal journeys of type Φ is a *temporal exact edge-cover* of G if every temporal edge $(e, t) \in \mathcal{E}$ appears in exactly one journey $J_i \in \mathcal{J}$.

Problem: (Non-)Strict Temporal Φ Exact Edge-Cover

Input: A temporal graph $G = \langle (V, E), \mathcal{E} \rangle$ and an integer $k \in \mathbb{N}$.

Question: Does there exist a temporal exact edge-cover of G by at most k journeys of type Φ , where each journey is non-strict or strict.

Results

Complexity Results

- For $k = 1$ *all* variants are solvable in polynomial time.
- For $k = 2$ the *directed and undirected non-strict temporal trail* variant is NP-Complete and the rest are solvable in polynomial time.
- For $k = 3$ the *directed strict temporal walk* variant is solvable in polynomial time and the rest are NP-Complete

		Path	Trail	Walk
Directed	Strict	NP-complete	NP-complete	P
	Non-Strict	NP-complete	NP-complete	W[2]-Hard
Undirected	Strict	NP-complete	NP-complete	NP-C & XP
	Non-Strict	NP-complete	NP-complete	NP-C & XP

The table summarizes the case of $k \geq 3$

All variants of the problem are placed in complexity classes using the following techniques:

1. Polynomial algorithm design (to show $\in \mathbf{P}$).
2. Cook Reduction $A \leq_T B$ (to show $A \in \mathbf{P}$).
3. Karp Reduction $A \leq_p B$ (to show $A \in \mathbf{P}$ or B is **NP**-hard).
4. FPT Redcution $A \leq_{FPT} B$.

Informal Definition (Polynomial-Time Reductions)

- **Turing / Cook (algorithmic) reduction:** an algorithm for A that runs in polynomial time while making polynomially many calls to a black-box solver for B . Think: “call a B -subroutine repeatedly and combine the results”.
- **Karp / many-one reduction:** a polynomial-time computable function f that maps instances x of A to instances $f(x)$ of B such that x is a yes-instance of A iff $f(x)$ is a yes-instance of B .

Strict-Time, Single Journey on Directed Graphs

Greedy Algorithm:

Input: The edgestream (edges in increasing time order).

0. Feasibility: if $\exists t$ with $|E_t| \geq 2$ then reject.
1. Pick the earliest temporal edge $(e, t_{\min})$; its orientation fixes the start.
2. Sweep edges in increasing time, keep current endpoint v and last time t_{prev} :
 - i) For each (f, t) append iff $t > t_{\text{prev}}$ and f is outgoing from v ; update v, t_{prev} .
 - ii) If some edge cannot be appended when required, the attempt fails.
3. Accept iff the single greedy journey covers every temporal edge exactly once.
 - Complexity: linear $O(|V| + |\mathcal{E}|)$
 - For undirected graphs: try both endpoints of the first edge as a starting vertices.

Strict Time, k -Temporal-Walk on Directed Graphs

Algorithm:

Input: The edgestream (edges in increasing time order).

Greedy - Streaming algorithm

```
1: For all  $x \in V$ :  $c[x] \leftarrow 0$            ▷ number of walks currently ending at  $x$ 
2:  $\text{starts} \leftarrow 0$ 
3: for each temporal edge  $(u \rightarrow v, t)$  in increasing  $t$  do
4:   if  $c[u] > 0$  then
5:      $c[u] \leftarrow c[u] - 1$            ▷ extend an existing walk ending at  $u$ 
6:   else
7:      $\text{starts} \leftarrow \text{starts} + 1$            ▷ start a new walk at  $u$ 
8:   end if
9:    $c[v] \leftarrow c[v] + 1$            ▷ the walk now ends at  $v$ 
10: end for
```

The actual walks can be constructed by tracking which walk is extended at each step.

Definition: Eulerian Paths/Trail

- *Eulerian trail / path*: a trail that uses every edge of the graph exactly once.
- *Eulerian cycle / circuit*: an Eulerian trail that starts and ends at the same vertex.

We know that:

1. There exists a polynomial-time algorithm that decides whether a given graph (directed or undirected) admits an Eulerian cycle or path, and that identifies the possible start/end vertices when a trail (non-closed) exists.
2. The algorithm runs in Polynomial (linear - $O(|V| + |E|)$) time (e.g., modified Hierholzer's algorithm).

Non-Strict Time, Single Temporal Trail on (Un)directed Graphs

Algorithm:

Input: The graph sequence.

1. **For each** time-slice $t = 1, \dots, t_{\max}$: compute the set P_t of feasible $(start, end)$ pairs for a traversal that uses every edge in E_t exactly once, under the chosen journey type Φ and orientation (directed/undirected) using the *Eulidian Journey Algorithm* .

2. **Dynamic chaining:** propagate reachability of endpoints across slices.

Initialise $S_1 = \{e_1 : (s_1, e_1) \in P_1\}$ (or restrict by fixed start terminal if given).

For $t = 2, \dots, t_{\max}$ update $S_t = \{e_t \mid (s_t, e_t) \in P_t \text{ and } s_t \in S_{t-1}\}$.

3. **Decide:** accept iff $S_{t_{\max}}$ is non-empty (there exists a compatible sequence of (s_t, e_t) giving one global journey that covers every temporal edge).

- **Complexity remark:** each slice is processed once and chaining is a simple propagation — overall polynomial time.

Strict Time, 2-Temporal-Paths in Directed Graphs

We show that the $k = 2$, strict time, temporal-path exact edge-cover in directed graphs can be decided by a Karp Reduction to an XOR(2)SAT.

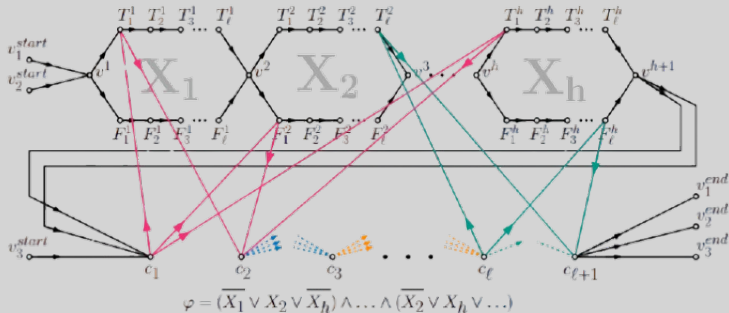
Reduction idea (very compact)

- Sweep edges in increasing time, maintaining two prefix paths with endpoints and visited sets.
- Attach forced edges deterministically.
- At an ambiguous hub (both prefixes can take an edge) create a Boolean variable recording which path takes it.
- Emit binary clauses whenever a combination of choices would cause a revisit or impossible attachment.
- The resulting XOR(2)SAT formula is satisfiable iff two valid strict temporal paths exist.

Strict Time, $(k \geq 3)$ -Temporal Path on Directed Graphs

The strict temporal path exact edge-cover problem on directed temporal graphs is *NP-hard*.

Proof idea. The hardness is shown by a polynomial-time (Karp) reduction from *NAE(k)SAT*.



Non-Strict, ($k \geq 3$)-Temporal-Walk, on Directed Graphs

To prove that the temporal exact edge-cover problem with non-strict walks on directed temporal graphs is NP-complete and W[2]-hard (when parameterized by the number of walks), we use a reduction from the *Hitting Set* problem.

Proof. Given a *Hitting Set* instance

$(N = \{1, \dots, n\}, \mathcal{S} = \{S_1, \dots, S_m\}, k)$, assume that all sets have size 2 or larger, otherwise augment them with auxiliary elements. We construct a temporal graph $G = \langle (V, E), \mathcal{E} \rangle$ with $V = N$. For every set $S_j = \{x_1, \dots, x_{|S_j|}\}$, we construct a cycle between its elements where all edges have time label j , i.e., for each $0 < i < |S_j|$ we add $((x_i, x_{i+1}), j)$, and lastly $((x_{|S_j|}, x_1), j)$.

Acknowledgments

Thanks for you attention 😊